Plan: Reach and benchmark Gömb to *break* signed-link AUC bars

HymeKo research track

2026-05-12

Scope and goal

Scope: signed link prediction with HymeKo-Gömb (`run_gomb_smoke`, `run_gomb_tune`) on the same datasets and `80_10_10` split already used for external SGCN / HSiKAN comparisons (Bitcoin Alpha/OTC, Slashdot, Epinions).

Goal: define a *publishable* procedure to (i) push mean test AUROC/AP beyond agreed reference bars, and (ii) report uncertainty and significance so "breaking" a metric is not a single-seed cherry-pick. **"Break"** means: mean test AUROC (or AP, if primary for a venue) exceeds the reference by more than a pre-declared margin, with a two-sided 95% bootstrap CI (or DeLong-style AUC comparison) that excludes equality.

Reference bars (locked targets)

- **Published SGCN** (Derr et al., SNAP Bitcoin graphs): AUROC ∼0.91–0.93 depending on graph (see `docs/plans/2026-05-12-gomb-external-auc-tuning/plan.tex`).
- **In-repo tuned SGCN** (draft/paper notes): e.g. Slashdot ∼0.914 under matched protocol; Bitcoin Alpha/OTC bars as in `PAPER_DRAFT_2026_05_03.md` / external-auc tuning report.
- **Current Gömb joint** (single seed, 48 epochs, compact): Slashdot `test_auroc`≈0.896 (JSONL `reports/gomb_tune_slashdot_joint_auc_2026_05_12.jsonl`). Gap to in-repo SGCN Slashdot ≈0.018 AUROC — primary stretch target.

CORE.YAML items touched

None for the benchmark harness and protocol documentation. Any future dependency pin (Optuna for Gömb, `scipy` for DeLong) is a Section 1 escalation.

Affected files (when implemented)

- **New:** `signedkan_wip/experiments/gomb_auc_break_bench.sh` (or Python driver) — orchestrates multi-seed reruns from frozen `trial_params`.
- **New:** `signedkan_wip/src/aggregate_gomb_jsonl.py` (optional) — median/IQR over seeds, peak RSS from `/usr/bin/time -v` or cgroup wrapper.
- **Append:** `reports/2026-05-12-gomb-auc-break-benchmark-results.md` — table: seed, test_auroc, test_ap, n_params, wall, peak RSS.
- **Optional:** extend `run_gomb_tune` with `--replay-params-json` to avoid hand-building CLI from JSON (not required for v1: shell can call `run_gomb_smoke` with explicit flags).

Interface changes

None at start. If `--replay-params-json` is added later: single entry point, one JSON schema — no Cartesian per-flag wrappers (CLAUDE.md §6.5).

Protocol (anti-leakage)

1. **Hyperparameter search** uses only **validation** metrics (`val_auroc`, `val_average_precision`) to rank trials during `run_gomb_tune`. The held-out `test_*` fields are logged but **not** used for early stopping or search decisions in the strict mode.
2. **Winner selection:** after search, fix `trial_params` for the best *val* row (not best test).
3. **Generalisation phase:** rerun that configuration for `data_seed` $\in \{0, \ldots, K-1\}$ with $K \geq 5$ (CLAUDE.md benchmark stability). Report **mean, std, median, IQR** of `test_auroc` and `test_average_precision`.
4. **Single-seed headline** numbers are diagnostic only; the headline claim uses the multi-seed aggregate.

Staged compute plan

Stage A — Epoch / convergence curve (single seed). Hold `trial_params` fixed (e.g. best val from a dense search). Sweep `n_epochs` $\in \{32, 48, 64, 96, 128\}$ (or until val plateaus). Record `val_auc_best`, `test_auroc` at final epoch. **Budget:** $\leq 10\%$ of full multi-seed wall before queuing overnight.

Stage B — Dense random / Sobol search (val objective). `run_gomb_tune` with ≥ 48 trials per dataset envelope (joint-mix SNAP compact; Bitcoin joint with existing clamps). Log `n_params`, inference throughput (already in smoke JSON).

Stage C — Multi-seed evaluation. For each dataset, one frozen config from Stage B; K seeds; optional `joint_slot_cap` ablation $\{8000, 12000, 16000, 0\}$ on SNAP if VRAM allows `0`.

Stage D — Significance vs bar. Either: (i) paired bootstrap on per-edge scores exported once (storage-heavy), or (ii) DeLong variance for AUROC if a reference model's per-edge scores exist; if not, report bootstrap over K seeds only (weaker claim: "mean exceeds bar by δ with CI not overlapping bar").

Test strategy

- Unit: JSONL aggregation script on synthetic 5-line fixture (median, IQR deterministic).
- Integration: one CPU smoke with two `data_seed` values and identical `trial_params` hash in logs.

Performance budget

Peak RSS $\leq$ 16 GB per subprocess (CLAUDE.md §4). Slashdot joint remains joint-slot subsampled by default; any increase in `topk` or removal of cap requires a one-seed VRAM smoke before multi-seed.

Worst-case inputs

Full Bitcoin joint tuple pools $\times$ wide `topk`; Slashdot without `joint_slot_cap`; Epinions $|V|$ embedding + four slots. All capped by existing smoke flags unless explicitly escalated.

Rollback

Delete orchestration script and aggregate helper; keep JSONL artifacts read-only for history.

Risk anticipation

Test leakage: repeated peeking at `test_auroc` during manual search invalidates claims — mitigated by val-only selection + frozen config. **Subsampling bias:** `joint_slot_cap` changes the training distribution; document cap in every table row. **Variance:** high variance across `data_seed` may prevent "break" despite a lucky single seed — mitigated by $K \geq 5$ and reporting IQR.

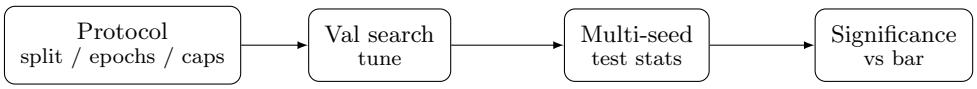

Figure 1: Stages from frozen protocol to defensible “break” claim.